

```
from xml.dom.minidom import parse, parseString
```

```
dom = parse('tvguide-ds.xml')
```

8.1 a)

```
# finding all movies
root = dom.documentElement
movies = dom.getElementsByTagName("movie")
movies_count = len(movies)
print("Total number of movies: " + str(movies_count))
```

```
↗ Total number of movies: 98
```

```
for movie in movies:
    movie1 = movie.getElementsByTagName("name")[0].firstChild.data
    print(movie1)
```

```
↗ Spaceballs
```

```
# Extracts all details of a movie
def get_details(movie):
    movie_data = {}

    for node in movie.childNodes:
        if node.nodeType == node.ELEMENT_NODE:
            tag = node.tagName

            # Check for attributes
            attributes = {attr: node.getAttribute(attr) for attr in node.attributes.keys()} if node.hasAttributes() else None

            # Get text values
            child_values = [child.nodeValue.strip() for child in node.childNodes if child.nodeType == child.TEXT_NODE and child.nodeValue]

            # Check for nested elements
            nested_data = get_details(node) if any(child.nodeType == node.ELEMENT_NODE for child in node.childNodes) else None

            # Store data appropriately
            if tag in movie_data:
                if not isinstance(movie_data[tag], list):
                    movie_data[tag] = [movie_data[tag]]
                movie_data[tag].append(nested_data if nested_data else " ".join(child_values))
            else:
                if nested_data:
                    if child_values:
                        nested_data["value"] = " ".join(child_values)
                        movie_data[tag] = nested_data
                    elif child_values:
                        movie_data[tag] = child_values[0] if len(child_values) == 1 else child_values

            if attributes:
                movie_data[f"{tag}_attributes"] = attributes

    return movie_data
```

```
get_details(movies[3])
```

```
↗
```

```
{ 'name': 'Rick Taggart', 'role': 'Costume Designer'},
{'name': 'Todd Gilbert', 'role': 'Assoc. Producer'},
{'name': 'Danny Dimbort', 'role': 'Exec. Producer'},
{'name': 'Gary Scott Thompson', 'role': 'Producer'},
{'name': 'Mary McLeod', 'role': 'Costumes'},
{'name': 'Margaret Yaworski', 'role': 'Make Up'},
{'name': 'Denis Lenoir', 'role': 'Cinematographer'},
{'name': 'Jon Avnet', 'role': 'Producer'},
{'name': 'Gerd Koechlin', 'role': 'Co-Producer'},
{'name': 'Jeremy Stanbridge', 'role': 'Art Director'},
{'name': 'Marsha Oglesby', 'role': 'Co-Producer'},
{'name': 'Bill Mills', 'role': 'Special Effects'},
{'name': 'Randall Emmett', 'role': 'Producer'}}],
'review-rating': '1',
'review': {'paragraph': ["Gary Scott Thompson and Jon Avnet's twisty tale of a forensic psychiatrist given 88 minutes to live
by a shadowy sociopath might have made an entertainingly direct-to-DVD B-movie thriller with the courage of its sleazy
convictions. But expectations are different for a theatrical feature starring Al Pacino, and by those standards it's a
preposterous misfire.",
'Nine years ago, Jack Gramm\'s (Pacino) testimony helped put Jon Forster (Neal McDonough) on death row for the torture
killing of Joanie Cates (Vicky Huang). Dubbed the "Seattle Slayer," Forster always maintained his innocence, claiming both that
Gramm lied under oath and that forensic psychiatry is voodoo science. On the eve of Forster\'s execution, the Seattle police
discover a crime scene with all his trademarks: Is it the work of a copycat killer, or was Forster wrongly convicted? Worse
still, the victim was one of Gramm\'s Northwest Washington University students; the FBI has picked up on rumors that he was
sleeping with her, and the killer taped her pleading with him to admit that he falsified testimony. Shortly after, Gramm
receives a phone call telling him he\'s going to die in 88 minutes. Over the course of the next hour and a half, Gramm is
threatened, attacked and finds himself suspected of two gruesome murders. He becomes convinced that his tormentor must be
someone close to him who\'s inexplicably conspiring to help Forster get a stay of execution and maybe even a new trial. Could it
be Gramm\'s assistant, Shelly (Amy Brenneman)? Perhaps one of his students, like whip-smart Lauren (Leelee Sobieski) or sullen
Mike (Benjamin McKenzie)? How about his teaching assistant, Kim (Alicia Witt), or maybe her ex-husband (Stephen Moyer), who did
time at Walla Walla Prison -- where Forster is confined -- and has been lurking around a lot recently? Maybe Dean Johnson
(Deborah Kara Unger), who seems to resent Gramm, or the creepy campus security guard (Brendan Fletcher) fascinated by serial
killers? But as evidence against Gramm mounts, even his old friend, Agent Frank Parks (William Forsythe), begins to wonder
whether he\'s become a murderer.',
'i': ['86th minute', 'Maitland McDonagh'],
'a': 'leave a comment',
'a_attributes': {'href': 'http://movies.tvguide.com/88-minutes/review/293098#comments',
'class': 'obj-more-inline'}}}]}
```

b)

```
from xml.sax import parse
from xml.sax.handler import ContentHandler

class IMDBHandler(ContentHandler):
    def __init__(self):
        self.current_tag = None
        self.string_builder = []
        self.current_movie = {}
        self.movies = []
        self.movie_count = 0

    def startElement(self, name, attrs):
        self.current_tag = name
        if name == "movie":
            self.current_movie = {}
            self.movie_count += 1

            # Store movie attributes
            if attrs.getLength() > 0:
                self.current_movie["attributes"] = {attr: attrs.getValue(attr) for attr in attrs.getNames()}

    def endElement(self, name):
        if name == "movie":
            self.movies.append(self.current_movie)
        elif self.current_tag and self.string_builder:
            value = " ".join(self.string_builder).strip()

            # Store movie details
            if name in self.current_movie:
                if isinstance(self.current_movie[name], list):
                    self.current_movie[name].append(value)
                else:
                    self.current_movie[name] = [self.current_movie[name], value]
            else:
                self.current_movie[name] = value

        self.current_tag = None
        self.string_builder = []

    def characters(self, content):
        if self.current_tag and content.strip():
            self.string_builder.append(content.strip())

# Create handler instance and parse XML
handler = IMDBHandler()
```

```
try:
    parse("imdb-ds.xml", handler)
except Exception as e:
    print("Parsing stopped:", e)

print(f"Total movies parsed: {handler.movie_count}")
```

↗ Total movies parsed: 243856

```
handler.movies[985]
```

↗

```
{'attributes': {'imdbid': 'idm423755831520'},
 'title': 'Mistaken Identity',
 'genre': ['Comedy', 'Short'],
 'name': ['Douglas, Sean (I)',
 'George, Martin (I)',
 'Liebe, Christopher',
 'Max, Don (I)',
 'Page, Madden',
 'Romano, Joe (IV)',
 'Stasi, Matt',
 'White, Michael (XVIII)',
 'Green-Gaber, Renata',
 'Mauro, Christina'],
 'role': ['Crandall',
 'Griffen',
 'Dashing Gentleman',
 'Mr. Wonderful',
 'Interregator',
 'Bathroom Guy',
 'Lone Gunman',
 'Office Suit',
 'Margaret',
 'Carly'],
 'director': 'Green-Gaber, Renata',
 'location': 'Los Angeles, California, USA',
 'year': '2004'}
```

c)

```
#!pip install jellyfish
```

↗ Requirement already satisfied: jellyfish in /usr/local/lib/python3.11/dist-packages (1.1.0)

Double-click (or enter) to edit

```
from xml.dom.minidom import parse, parseString
import xml.dom.minidom as minidom
import xml.sax
import jellyfish
from itertools import product

# Parsing IMDB XML using SAX and extracts movie titles
class IMDBHandler_titles(xml.sax.ContentHandler):
    def __init__(self):
        self.current_tag = None
        self.title = None
        self.titles = []

    def startElement(self, name, attrs):
        self.current_tag = name
        if name == "movie":
            self.title = None # Reset title

    def endElement(self, name):
        if name == "title" and self.title:
            self.titles.append(self.title.strip())
        if len(self.titles) >= 5: # Stop after 5 titles
            raise Exception("Limit reached")

    def characters(self, content):
        if self.current_tag == "title":
            self.title = (self.title or "") + content

# for parsing the TV Guide
def parse_tv_guide(file):
    doc = minidom.parse(file)
```

```
movies = doc.getElementsByTagName("movie")
titles = [movie.getElementsByTagName("name")[0].firstChild.data for movie in movies[:5]]
return titles

#for parsing IMDB
def parse_imdb(file):
    handler = IMDBHandler_titles()
    try:
        xml.sax.parse(file, handler)
    except Exception:
        pass
    return handler.titles

def calculate_similarity(tv_titles, imdb_titles):
    def levenshtein(a, b):
        return jellyfish.levenshtein_distance(a, b)

    def jaro_winkler(a, b):
        return jellyfish.jaro_winkler_similarity(a, b)

    def hamming(a, b):
        return jellyfish.hamming_distance(a.ljust(max(len(a), len(b))), b.ljust(max(len(a), len(b))))

    def soundex_similarity(a, b):
        soundex_a = jellyfish.soundex(a)
        soundex_b = jellyfish.soundex(b)
        jw_score = jaro_winkler(a, b) # we chose to use Jaro-Winkler as extra measure
        return jw_score if soundex_a == soundex_b else 0

    metrics = {
        "Levenshtein Distance": levenshtein,
        "Jaro-Winkler Similarity": jaro_winkler,
        "Hamming Distance": hamming,
        "Soundex": soundex_similarity
    }

    for metric_name, metric in metrics.items():
        print(f"\n {metric_name} Scores:")
        for tv, imdb in product(tv_titles, imdb_titles):
            score = metric(tv, imdb)
            print(f"- '{tv}' vs '{imdb}': {score}")

tv_titles = parse_tv_guide("tvguide-ds.xml")
imdb_titles = parse_imdb("imdb-ds.xml")

print("TV Guide Titles:", tv_titles)
print("IMDB Titles:", imdb_titles)

calculate_similarity(tv_titles, imdb_titles)
```



```

soundex scores:
- 'Ever After' vs 'Katka i Shiz': 0
- 'Ever After' vs 'Prisoner of Zenda': 0
- 'Ever After' vs 'Dough for the Do-Do': 0
- 'Ever After' vs 'Bad Credit and Aliens': 0
- 'Ever After' vs 'Texana cien X #3': 0
- 'First Daughter' vs 'Katka i Shiz': 0
- 'First Daughter' vs 'Prisoner of Zenda': 0
- 'First Daughter' vs 'Dough for the Do-Do': 0
- 'First Daughter' vs 'Bad Credit and Aliens': 0
- 'First Daughter' vs 'Texana cien X #3': 0
- 'Hope Floats' vs 'Katka i Shiz': 0
- 'Hope Floats' vs 'Prisoner of Zenda': 0
- 'Hope Floats' vs 'Dough for the Do-Do': 0
- 'Hope Floats' vs 'Bad Credit and Aliens': 0
- 'Hope Floats' vs 'Texana cien X #3': 0
- '88 Minutes' vs 'Katka i Shiz': 0
- '88 Minutes' vs 'Prisoner of Zenda': 0
- '88 Minutes' vs 'Dough for the Do-Do': 0
- '88 Minutes' vs 'Bad Credit and Aliens': 0
- '88 Minutes' vs 'Texana cien X #3': 0
- 'Chicago' vs 'Katka i Shiz': 0
- 'Chicago' vs 'Prisoner of Zenda': 0
- 'Chicago' vs 'Dough for the Do-Do': 0
- 'Chicago' vs 'Bad Credit and Aliens': 0
- 'Chicago' vs 'Texana cien X #3': 0

```

d)

```
threshold = 0.85
```

```
output_file = "matched_movies.txt"
```

```
def jaro_winkler(a, b):
    return jellyfish.jaro_winkler_similarity(a, b)
```

```

class IMDBHandler_d(xml.sax.ContentHandler):
    def __init__(self):
        self.current_tag = None
        self.title = None
        self.titles = []

    def startElement(self, name, attrs):
        self.current_tag = name
        if name == "movie":
            self.title = None # Reset title

    def endElement(self, name):
        if name == "title" and self.title:
            self.titles.append(self.title.strip()) # Store title

    def characters(self, content):
        if self.current_tag == "title":
            self.title = (self.title or "") + content # Store movie title

```

```

def parse_imdbb(file):
    handler = IMDBHandler_d()
    xml.sax.parse(file, handler)
    return handler.titles

```

```

def parse_tv_guide_all(file):
    doc = minidom.parse(file)
    movies = doc.getElementsByTagName("movie")
    titles = [movie.getElementsByTagName("name")[0].firstChild.data for movie in movies]
    return titles

```

```
def find_matches(tv_titles, imdb_titles, threshold, output_file):
```

```

    with open(output_file, "w", encoding="utf-8") as f:
        for tv_title in tv_titles:
            best_match = None
            best_score = 0

            for imdb_title in imdb_titles:
                score = jaro_winkler(tv_title, imdb_title)

                if score > best_score:
                    best_score = score
                    best_match = imdb_title

            if best_score >= threshold:
                f.write(f"{tv_title} -> {best_match} (Score: {best_score:.4f})\n")

```

```
print("see matching in the output file")

tv_titles = parse_tv_guide_all("tvguide-ds.xml")
imdb_titles = parse_imdbb("imdb-ds.xml")

find_matches(tv_titles, imdb_titles, threshold, output_file)

↗ see matching in the output file
```

e)

```
from collections import defaultdict
def find_best_unique_matches(tv_titles, imdb_titles, threshold, output_file):
    match_candidates = []

    # find the best match for each TV movie
    for tv_title in tv_titles:
        best_match = None
        best_score = 0

        for imdb_title in imdb_titles:
            score = jaro_winkler(tv_title, imdb_title)

            if score > best_score:
                best_score = score
                best_match = imdb_title

        if best_score >= 0.5:
            match_candidates.append((tv_title, best_match, best_score))

    # make sure imdb movies are assigned uniquely
    assigned_imdb = set()
    final_matches = []

    # Sort matches by score
    match_candidates.sort(key=lambda x: x[2], reverse=True)

    for tv_title, imdb_title, score in match_candidates:
        if imdb_title not in assigned_imdb:
            final_matches.append((tv_title, imdb_title, score))
            assigned_imdb.add(imdb_title)

    # write final matches to file
    with open(output_file, "w", encoding="utf-8") as f:
        for tv_title, imdb_title, score in final_matches:
            f.write(f"{tv_title} -> {imdb_title} (Score: {score:.4f})\n")

    print("Unique best matches written to output file")

tv_titles = parse_tv_guide_all("tvguide-ds.xml")
imdb_titles = parse_imdbb("imdb-ds.xml")

find_best_unique_matches(tv_titles, imdb_titles, threshold, "unique_matches.txt")

↗ Unique best matches written to output file
```

8.2) a)

```
import jellyfish
from collections import defaultdict

# Function to compute different similarity scores
def compute_similarity(tv_title, imdb_title, method):
    if method == "jaro_winkler":
        return jellyfish.jaro_winkler_similarity(tv_title, imdb_title)
    elif method == "levenshtein":
        max_len = max(len(tv_title), len(imdb_title))
        return 1 - (jellyfish.levenshtein_distance(tv_title, imdb_title) / max_len)
    elif method == "hamming":
        if len(tv_title) != len(imdb_title):
            return 0
        return 1 - (jellyfish.hamming_distance(tv_title, imdb_title) / max(len(tv_title), len(imdb_title)))
    elif method == "soundex":
        soundex_a = jellyfish.soundex(tv_title)
        soundex_b = jellyfish.soundex(imdb_title)
        jw_score = jellyfish.jaro_winkler_similarity(tv_title, imdb_title) # we chose to use Jaro-Winkler as extra measure
        return jw_score if soundex_a == soundex_b else 0
    else:
```

```

    raise ValueError("Invalid similarity method")

def find_best_unique_matches(tv_titles, imdb_titles, threshold, output_file_prefix):
    similarity_methods = ["levenshtein", "jaro_winkler", "hamming", "soundex"]
    for method in similarity_methods:
        match_candidates = [] # To store all possible matches for each similarity function

        # find the best match for each TV movie
        for tv_title in tv_titles:
            best_match = None
            best_score = 0

            for imdb_title in imdb_titles:
                score = compute_similarity(tv_title, imdb_title, method)

                if score > best_score:
                    best_score = score
                    best_match = imdb_title

            if best_score >= threshold:
                match_candidates.append((tv_title, best_match, best_score))

    assigned_imdb = set()
    final_matches = []

    for tv_title, imdb_title, score in match_candidates:
        if imdb_title not in assigned_imdb:
            final_matches.append((tv_title, imdb_title, score))
            assigned_imdb.add(imdb_title)

    # Write final matches to different files based on similarity function
    output_file = f"{output_file_prefix}_{method}_matches.txt"
    with open(output_file, "w", encoding="utf-8") as f:
        for tv_title, imdb_title, score in final_matches:
            f.write(f"{tv_title},{imdb_title},{score:.4f}\n")

    print(f"Matches using {method} similarity saved to {output_file}")

tv_titles = parse_tv_guide_all("tvguide-ds.xml")
imdb_titles = parse_imdbb("imdb-ds.xml")

threshold = 0.0 # Adjust the threshold => 0 for now because we wanted to see all TV Guide movies matched
find_best_unique_matches(tv_titles, imdb_titles, threshold, "unique_matches")

#function to separate IMDB movies from matching

def resultIMDBtitles(path):
    f = open(path, "r")
    titlesIMDB = []
    for line in f.readlines():
        titlesIMDB.append(line.strip().split(",")[1])
    return titlesIMDB

titles_hamming = resultIMDBtitles("/content/unique_matches_hamming_matches.txt")
titles_jaro = resultIMDBtitles("/content/unique_matches_jaro_winkler_matches.txt")
titles_lev = resultIMDBtitles("/content/unique_matches_levenshtein_matches.txt")
titles_soundex = resultIMDBtitles("/content/unique_matches_soundex_matches.txt")

#function to get IDs for IMDB titles

import xml.etree.ElementTree as ET

def parse_imdb_ids(file_path):
    imdb_ids = {}

    tree = ET.parse(file_path)
    root = tree.getroot()

    for movie in root.findall("movie"):
        imdbid = movie.attrib.get("imdbid")
        title_element = movie.find("title")

        if imdbid and title_element is not None:
            title = title_element.text.strip()

```

```

imdb_ids[title] = imdbid

return imdb_ids

def get_imdb_ids_for_titles(imdb_titles, imdb_mapping):
    """Finds IMDB IDs for a given list of movie titles."""
    result = [imdb_mapping.get(title, "Unknown") for title in imdb_titles]
    return result

imdb_mapping = parse_imdb_ids("imdb-ds.xml")

print(titles_hamming)

[ 'Ever After', 'First Daughter', 'Hope Floats', '88 Minutes', 'Chicago', 'All the King's Men', 'Across the Universe', 'XX.', 'Outlaw

result_hamming = get_imdb_ids_for_titles(titles_hamming, imdb_mapping)
result_jaro = get_imdb_ids_for_titles(titles_jaro, imdb_mapping)
result_levenshtein = get_imdb_ids_for_titles(titles_lev, imdb_mapping)
result_soundex = get_imdb_ids_for_titles(titles_soundex, imdb_mapping)
print(len(result_hamming))

97

#padding the prediction list so that we can use the sklearn evaluation function
def pad_predictions(y_true, y_predict):
    padded_y_predict = []

    for true_value in y_true:
        if true_value in y_predict:
            padded_y_predict.append(true_value)
        else:
            padded_y_predict.append('0') # Pad missing values

    return padded_y_predict

from sklearn.metrics import precision_recall_fscore_support

relevant = []

with open ("ground_truth-1.txt","r") as f:
    for line in f.readlines():
        relevant.append(line.strip().split(",")[1])

results = [result_hamming, result_jaro, result_levenshtein, result_soundex]

for result in results:
    resultT = pad_predictions(relevant, result)
    preHamming,recHamming,f1Hamming,avHamming = precision_recall_fscore_support(relevant, resultT, average="micro")
    print("Precision: ",preHamming,"\nRecall:",recHamming,"\nF1 score:",f1Hamming,"\n")
    resultT = []

Precision: 0.4387755102040816
Recall: 0.4387755102040816
F1 score: 0.4387755102040816

Precision: 0.5204081632653061
Recall: 0.5204081632653061
F1 score: 0.5204081632653061

Precision: 0.5
Recall: 0.5
F1 score: 0.5

Precision: 0.5
Recall: 0.5
F1 score: 0.5

```

✓ b) Adding year and actors to matching

```

#parsing the tv guide, but now with extra attributes

def parse_tv_guide_all_updated(file):
    doc = minidom.parse(file)
    movies = doc.getElementsByTagName("movie")

    movie dict = {} # {title: (id, year, set(actors))}

```

```

for movie in movies:
    tv_id = movie.getAttribute("tvgid")
    name_nodes = movie.getElementsByTagName("name")
    year_nodes = movie.getElementsByTagName("year")
    actor_nodes = movie.getElementsByTagName("actor")

    title = name_nodes[0].firstChild.data.strip() if name_nodes else None
    year = int(year_nodes[0].firstChild.data.strip()) if year_nodes else None
    actors = {actor.getElementsByTagName("name")[0].firstChild.nodeValue.strip()
               for actor in actor_nodes
               if actor.getElementsByTagName("name") and actor.getElementsByTagName("name")[0].firstChild}

    if title:
        movie_dict[tv_id] = (title, year, actors)

return movie_dict

import xml.sax
from xml.sax.handler import ContentHandler

class IMDBHandlerUpdated(ContentHandler):
    def __init__(self):
        super().__init__()
        self.current_tag = None
        self.current_id = None
        self.current_title = None
        self.current_year = None
        self.current_actors = set()
        self.movie_dict = {}
        self.content_buffer = ""
        self.in_movie = False

    def startElement(self, name, attrs):
        self.current_tag = name
        if name == "movie":
            self.in_movie = True
            if attrs.get("imdbid"):
                self.current_id = attrs.get("imdbid")
            self.current_title = None
            self.current_year = None
            self.current_actors = set()

            self.content_buffer = ""

    def endElement(self, name):
        if self.in_movie:
            if name == "title":
                self.current_title = self.content_buffer.strip()
            elif name == "year":
                try:
                    self.current_year = int(self.content_buffer.strip())
                except (ValueError, TypeError):
                    self.current_year = None
            elif name == "name":
                if self.content_buffer.strip():
                    self.current_actors.add(self.content_buffer.strip())

            elif name == "movie":
                if self.current_id and self.current_title:
                    # store in the format (title, year, {actors})
                    self.movie_dict[self.current_id] = (
                        self.current_title,
                        self.current_year,
                        self.current_actors
                    )
                self.in_movie = False

            # reset buffer and current tag
            self.content_buffer = ""
            if name == self.current_tag:
                self.current_tag = None

    def characters(self, content):
        if self.current_tag:
            self.content_buffer += content

    def get_movie_dict(self):
        return self.movie_dict

```

```

# Function to parse the IMDB XML file
def parse_imdb_act_year(file):
    handler = IMDBHandlerUpdated()
    xml.sax.parse(file, handler)
    return handler.get_movie_dict()

def year_similarity(tv_year, imdb_year):
    if tv_year is None or imdb_year is None:
        return 0 # no match if is missing
    return 1 if tv_year == imdb_year else max(0, 1 - (abs(tv_year - imdb_year) / 10)) # we made a small penalty if the years are close

def jaccard_similarity(tv_actors, imdb_actors):
    if not tv_actors or not imdb_actors:
        return 0
    intersection = len(tv_actors & imdb_actors)
    union = len(tv_actors | imdb_actors)
    return intersection / union if union > 0 else 0

def compute_final_similarity(tv_data, imdb_data, method, weights):
    tv_title, tv_year, tv_actors = tv_data
    imdb_title, imdb_year, imdb_actors = imdb_data

    title_sim = compute_similarity(tv_title, imdb_title, method)
    year_sim = year_similarity(tv_year, imdb_year)
    actor_sim = jaccard_similarity(set(tv_actors), set(imdb_actors))

    # Weighted sum
    final_score = (
        weights["title"] * title_sim +
        weights["year"] * year_sim +
        weights["actors"] * actor_sim
    )

    return final_score

def find_best_unique_matches_actorsyears(tv_data, imdb_data, threshold, output_file, weights):
    match_candidates = []

    for tv_entry in tv_data:
        tv_title, tv_year, tv_actors = tv_entry
        best_match = None
        best_score = 0

        for imdb_entry in imdb_data:
            imdb_title, imdb_year, imdb_actors = imdb_entry

            # compute final similarity score using Jaro-Winkler (it produced the best result before)
            score = compute_final_similarity(
                (tv_title, tv_year, tv_actors),
                (imdb_title, imdb_year, imdb_actors),
                method="jaro_winkler",
                weights=weights
            )

            if score > best_score:
                best_score = score
                best_match = imdb_title # store just the IMDb title

        if best_score >= threshold and best_match:
            match_candidates.append((tv_title, best_match, best_score)) # (tv_title, imdb_title, score)

    # keep original TV guide order when assigning unique IMDb movies
    assigned_imdb_titles = set()
    final_matches = []

    for tv_title, imdb_title, score in match_candidates:
        if imdb_title not in assigned_imdb_titles:
            final_matches.append((tv_title, imdb_title, score))
            assigned_imdb_titles.add(imdb_title)

    # Write results to file
    with open(output_file, "w", encoding="utf-8") as f:
        for tv_title, imdb_title, score in final_matches:
            f.write(f"{tv_title},{imdb_title},{score:.4f}\n")

    print(f"Matches saved to {output_file}")
    return final_matches

```

```
from itertools import islice
```

```
tv_data = parse_tv_guide_all_updated("tvguide-ds.xml") # Extract (title, year, actors)
print(tv_data)
```

```
imdb_data = parse_imdb_act_year("imdb-ds.xml") # Extract (title, year, actors, imdb_id)
print(dict(islice(imdb_data.items(), 2)))
```

```
{'2': ('Ever After', 1998, {'Jeanne Moreau', 'Mark Lewis', 'Dominic Rold', 'Matyelok Gibbs', 'Ricardo Cruz', 'Anjelica Huston', 'Dou  
{'idm423783729296': ('Katka i Shiz', 1992, {'Dzhigarkhanyan, Armen', 'Vasilyev, Yuri (I)', 'Karasyov, Denis', 'Megvinetukhutsesi, Ot
```

```
weights = {"title": 0.4, "year": 0.4, "actors": 0.2}
```

```
find_best_unique_matches_actorsyears(tv_data.values(), imdb_data.values(), threshold=0.5, output_file="matches_jaro.txt", weights=weights)
```

```
{'Hellboy', 'Hellboy', 0.8),  
( 'Miracle At St. Anna', 'Malek', 0.6680701754385965),  
( 'The Devil Wears Prada', 'Thieves', 0.7136507936507936),  
( 'Australia', 'Australia', 0.8),  
( 'Role Models', 'Stolen Moments', 0.7064935064935065),  
( 'The Searchers', 'Three Bad Sisters', 0.7071548059783355),  
( 'Eagle Eye', 'Battle Royale', 0.6837606837606838),  
( 'Atl', 'Atemlos', 0.7390476190476192),  
( 'The Boondock Saints', 'Second Skin', 0.6980542264752791),  
( 'Live Free Or Die Hard', 'Live Free or Die Hard', 0.7923809523809524),  
( 'Tropic Thunder', 'Trophy', 0.7352380952380952),  
( 'Transporter 2', 'Transporteur II, Le', 0.7643724696356275),  
( 'Exit Wounds', 'Exit Wounds', 0.8034188034188035),  
( 'Towelhead', 'Howl', 0.6777777777777778),  
( 'Bolt', 'Bolden!', 0.7233333333333334),  
( 'Men In Black II', 'Men in Black II', 0.7893333333333334),  
( 'Four Christmases', 'Four Christmases', 0.76),  
( 'Virtual Jfk: Vietnam If Kennedy Had Lived',  
'Minutemen',  
0.6612014453477868),  
( 'The World is Not Enough', 'World Is Not Enough, The', 0.6858759216882786),  
( "Pirates Of The Caribbean: Dead Man's Chest",  
'Pirates of the Caribbean: Dead Man's Chest',  
0.7923809523809524),  
( 'Sex And The City', 'Sex and Death 101', 0.6922192513368985),  
( 'Transporter 3', 'Trap Door', 0.7186324786324787),  
( 'Out of Sight', 'Out of Sight', 0.8),  
( 'Victor/Victoria', 'Victor/Victoria', 0.8),  
( 'Waitress', 'Waitress', 0.8),  
( "Can't Hardly Wait", "Can't Hardly Wait", 0.8),  
( 'W.', 'W.C.', 0.7066666666666668),  
( 'Vanilla Sky', 'Vanilla Sky', 0.8),  
( 'Titanic', 'Titanic', 0.8),  
( 'Twilight', 'Til Night', 0.7183333333333334),  
( 'The Perfect Nanny', 'Terc', 0.7082352941176471),  
( 'Halloween', 'Halloween', 0.8),  
( 'Meet Me In St. Louis', 'Meet Me in St. Louis', 0.792),  
( 'Zack And Miri Make A Porno', 'Bran Mak Morn', 0.6735042735042736),  
( 'Star Trek Iv: The Voyage Home',  
'Star Trek IV: The Voyage Home',  
0.7859113300492612),  
( 'Wall-E', 'Wall-E', 0.76),  
( 'Iron Man', 'Iron Man', 0.8),  
( 'The Benchwarmers', 'Thicker Than Water', 0.7057912457912459),  
( 'Saw V', 'Saw IV', 0.7466666666666668),  
( 'The Godfather', 'Godfather, The', 0.6903096903096904),  
( 'The Dark Knight', 'Taken', 0.696),  
( 'Fantastic Four', 'Fantastic Four', 0.8),  
( "Tyler Perry's The Family That Preys", 'Terra', 0.6971428571428572),  
( 'Come Early Morning', 'Come Early Morning', 0.8),  
( 'Jawbreaker', 'Jawbreaker', 0.8),  
( 'Frozen River', 'Frenemies', 0.7111111111111111),  
( 'The Boy In The Striped Pajamas',  
'Boy in the Striped Pajamas, The',  
0.709434794596085),  
( 'Body Of Evidence', 'Body of Evidence', 0.79),  
( 'Burn After Reading', 'Burn After Reading', 0.8),  
( 'Spaceballs', 'Spaceballs', 0.8)]
```

```
titles_jaro_actyear = resultIMDBtitles("/content/matches_jaro.txt")
print(titles_jaro_actyear)
```

```
['Ever After', 'First Daughter', 'Hope Floats', '88 Minutes', 'Chicago', "All the King's Men", 'Across the Universe', 'XX/XY', 'Quar
```

```
result_jaro_actyear = get_imdb_ids_for_titles(titles_jaro_actyear, imdb_mapping)
print(result_jaro_actyear)
```

```
['idm425250523376', 'idm424963555104', 'idm425290058512', 'idm424707838832', 'idm425129366640', 'idm424749022656', 'idm424475490592
```

```

from sklearn.metrics import precision_recall_fscore_support

relevant = []

with open ("ground_truth-1.txt","r") as f:
    for line in f.readlines():
        relevant.append(line.strip().split(",")[1])

result = pad_predictions(relevant, result_jaro_actyear)
print(result)
pre,rec,f1,av = precision_recall_fscore_support(relevant, result, average="micro")
print("Precision: ",pre,"\nRecall:",rec,"\nF1 score:",f1)

[ 'idm425250523376', 'idm424963555104', 'idm425290058512', 'idm424707838832', '0', 'idm424749022656', 'idm424475490592', '0', '0', 'i
Precision:  0.4387755102040816
Recall: 0.4387755102040816
F1 score: 0.4387755102040816

```

8.3 Mapping

```

import re
def standardize_title(title):
    title = title.strip()
    match = re.match(r'(.+), (The|A|An)$', title, re.IGNORECASE)
    if match:
        title = f"{match.group(2)} {match.group(1)}"

    title = re.sub(r'\s+', ' ', title).strip()
    return title

def standardize_actor(actor):
    actor = actor.strip()

    if "," in actor:
        parts = [part.strip().capitalize() for part in actor.split(",")]
        actor = f"{parts[1]} {parts[0]}" if len(parts) > 1 else actor.capitalize()
    else:
        actor = " ".join(word.capitalize() for word in actor.split())

    return actor

def extract_imdb_movies_with_actors(imdb_data):
    imdb_movies = {}

    imdb_datakeys = list(imdb_data.keys())
    for movie_id in imdb_datakeys:
        movie = imdb_data.get(movie_id)
        title = standardize_title(movie[0])
        year = movie[1]
        actors = {standardize_actor(actor) for actor in movie[2]}

        if movie_id:
            imdb_movies[movie_id] = (title, year, actors)

    return imdb_movies

standard_names = extract_imdb_movies_with_actors(imdb_data)
print(dict(islice(standard_names.items(), 2)))

{ 'idm423783729296': ('Katka i Shiz', 1992, {'Georgi Drozd', 'Otar Megvinetukhutsesi', 'Armen Dzhigarkhanyan', 'Irina Metlitskaya',

```

```
( 'Transporter 2', 'Transporteur 11, le', 0.8001900514538094),
('Exit Wounds', 'Exit Wounds', 0.8056818181818182),
('Towelhead', 'The Evil Dead', 0.6008058608058608),
('Bolt', 'Bolder!', 0.6233333333333333),
('Men In Black II', 'Men in Black II', 0.7917723577235772),
('Four Christmases', 'Four Christmases', 0.67),
('Virtual JFK: Vietnam If Kennedy Had Lived',
'Minutemen',
0.5612014453477868),
('The World is Not Enough', 'The World Is Not Enough', 0.864472049689441),
('Pirates Of The Caribbean: Dead Man's Chest',
'Pirates of the Caribbean: Dead Man's Chest',
0.8143589743589743),
('Sex And The City', 'Sex and Death 101', 0.6022192513368985),
('Transporter 3', 'Transformers', 0.6253846153846154),
('Out of Sight', 'Out of Sight', 0.7927272727272727),
('Victor/Victoria', 'Victor/Victoria', 0.8117647058823529),
('Waitress', 'Waitress', 0.8105263157894737),
('Can't Hardly Wait', 'Can't Hardly Wait', 0.7999999999999999),
('W.', 'W.C.', 0.6166666666666667),
('Vanilla Sky', 'Vanilla Sky', 0.8057142857142857),
('Titanic', 'Titanic', 0.8125),
('Twilight', 'Til Night', 0.6283333333333334),
('The Perfect Nanny', 'The Perfect Nanny', 0.8063636363636364),
('Halloween', 'Halloween', 0.7774193548387096),
('Meet Me In St. Louis', 'Meet Me in St. Louis', 0.7444999999999999),
('Zack And Miri Make A Porno', 'Zara', 0.5797435897435899),
('Star Trek Iv: The Voyage Home',
'Star Trek IV: The Voyage Home',
0.7609113300492611),
('Wall-E', 'Wall-E', 0.67),
('Iron Man', 'Iron Man', 0.7),
('The Benchwarmers', 'The Benchwarmers', 0.7370786516853932),
('Saw V', 'Saw IV', 0.6566666666666667),
('The Godfather', 'The Godfather', 0.9025),
('The Dark Knight', 'The Dark Knight', 0.7094736842105263),
('Fantastic Four', 'Fantastic Four', 0.8189655172413792),
('Tyler Perry's The Family That Preys', 'Terra', 0.5971428571428572),
('Come Early Morning', 'Come Early Morning', 0.7738461538461539),
('Jawbreaker', 'Jawbreaker', 0.85),
('Frozen River', 'Green River', 0.6124242424242425),
('The Boy In The Striped Pajamas',
'The Boy in the Striped Pajamas',
0.677904761904762),
('Body Of Evidence', 'Body of Evidence', 0.7746153846153846),
('Burn After Reading', 'Burn After Reading', 0.7096774193548386),
('Soaceballs'. 'Soaceballs'. 0.8363636363636363))
```

```
titles_jaro_mapping = resultIMDBtitles("/content/matches_mapping.txt")
print(titles_jaro_mapping)
```

```
['Ever After', 'First Daughter', 'Hope Floats', '88 Minutes', 'Chicago', "All the King's Men", 'Across the Universe', 'XX/XY', 'Dual
```

```
result_jaro_mapping = get_imdb_ids_for_titles(titles_jaro_mapping, imdb_mapping)
print(result_jaro_mapping)
```

```
['idm425250523376', 'idm424963555104', 'idm425290058512', 'idm424707838832', 'idm425129366640', 'idm424749022656', 'idm424475490592
```

```
from sklearn.metrics import precision_recall_fscore_support
```

```
relevant = []
```

```
with open ("ground_truth-1.txt","r") as f:
    for line in f.readlines():
        relevant.append(line.strip().split(",")[1])
```

```
result = pad_predictions(relevant, result_jaro_mapping)
print(result)
pre,rec,f1,av = precision_recall_fscore_support(relevant, result, average="micro")
print("Precision: ",pre,"\nRecall:",rec,"\nF1 score:",f1)
```

```
['idm425250523376', 'idm424963555104', 'idm425290058512', 'idm424707838832', '0', 'idm424749022656', 'idm424475490592', '0', '0', 'i
Precision: 0.47959183673469385
Recall: 0.47959183673469385
F1 score: 0.47959183673469385
```

8.4) Merging

a, b, c together

```
import xml.sax
```

```

from xml.sax.handler import ContentHandler
import json
import ast
import re

class IMDBHandlerCompleteALL(ContentHandler):
    def __init__(self):
        super().__init__()
        self.current_tag = None
        self.current_id = None
        self.current_title = None
        self.current_year = None
        self.current_genre = []
        self.current_actors = []
        self.current_roles = []
        self.current_director = None
        self.current_location = None
        self.current_keywords = []
        self.current_plot = None
        self.movie_dict = {}
        self.content_buffer = ""
        self.in_movie = False
        self.in_actor = False

    def startElement(self, name, attrs):
        self.current_tag = name
        if name == "movie":
            self.in_movie = True
            if attrs.get("imdbid"):
                self.current_id = attrs.get("imdbid")
            self.current_title = None
            self.current_year = None
            self.current_genre = []
            self.current_actors = []
            self.current_roles = []
            self.current_director = None
            self.current_location = None
            self.current_keywords = []
            self.current_plot = None
        elif name == "actor":
            self.in_actor = True

        self.content_buffer = ""

    def endElement(self, name):
        if self.in_movie:
            if name == "title":
                self.current_title = self.content_buffer.strip()
            elif name == "year":
                try:
                    self.current_year = int(self.content_buffer.strip())
                except (ValueError, TypeError):
                    self.current_year = None
            elif name == "genre":
                if self.content_buffer.strip():
                    self.current_genre.append(self.content_buffer.strip())
            elif name == "director":
                self.current_director = self.content_buffer.strip()
            elif name == "location":
                self.current_location = self.content_buffer.strip()
            elif name == "keyword":
                if self.content_buffer.strip():
                    self.current_keywords.append(self.content_buffer.strip())
            elif name == "plot":
                self.current_plot = self.content_buffer.strip()
            elif name == "name" and self.in_actor:
                if self.content_buffer.strip():
                    self.current_actors.append(self.content_buffer.strip())
            elif name == "role" and self.in_actor:
                if self.content_buffer.strip():
                    self.current_roles.append(self.content_buffer.strip())
            elif name == "actor":
                self.in_actor = False

        elif name == "movie":
            if self.current_id and self.current_title:
                movie_data = {
                    'title': self.current_title,
                    'year': self.current_year,
                    'imdbid': self.current_id,
                    'genres': self.current_genre,
                    'director': self.current_director,
                    'location': self.current_location,

```

```

        'keywords': self.current_keywords,
        'plot': self.current_plot,
        'actors': []
    }

    for i in range(min(len(self.current_actors), len(self.current_roles))):
        movie_data['actors'].append({
            'name': self.current_actors[i],
            'role': self.current_roles[i]
        })

    self.movie_dict[self.current_id] = movie_data
    self.in_movie = False

    self.content_buffer = ""
    if name == self.current_tag:
        self.current_tag = None

    def characters(self, content):
        if self.current_tag:
            self.content_buffer += content

    def get_movie_dict(self):
        return self.movie_dict

from xml.dom import minidom

def get_text(element):
    return element.firstChild.nodeValue.strip() if element and element.firstChild and element.firstChild.nodeValue else None

# here we parse all of the attributes and saving them in a dictionary
def parse_tv_guide_ALL(file):
    try:
        doc = minidom.parse(file)
        movie_elements = doc.getElementsByTagName("movie")

        movie_dict = {} # {tv_id: movie_data}

        for movie in movie_elements:
            tv_id = movie.getAttribute("tvgid")
            if not tv_id:
                continue

            title = get_text(movie.getElementsByTagName("name")[0]) if movie.getElementsByTagName("name") else None
            year = get_text(movie.getElementsByTagName("year")[0]) if movie.getElementsByTagName("year") else None
            rating = get_text(movie.getElementsByTagName("rating")[0]) if movie.getElementsByTagName("rating") else None
            country = get_text(movie.getElementsByTagName("country")[0]) if movie.getElementsByTagName("country") else None
            running_time = get_text(movie.getElementsByTagName("running-time")[0]) if movie.getElementsByTagName("running-time") else None
            format_type = get_text(movie.getElementsByTagName("format")[0]) if movie.getElementsByTagName("format") else None
            review_rating = get_text(movie.getElementsByTagName("review-rating")[0]) if movie.getElementsByTagName("review-rating") else None

            genres = set()
            for genre in movie.getElementsByTagName("genre"):
                genre_text = get_text(genre)
                if genre_text:
                    genres.add(genre_text)

            production_companies = set()
            for prod in movie.getElementsByTagName("production-company"):
                prod_text = get_text(prod)
                if prod_text:
                    production_companies.add(prod_text)

            released_by = get_text(movie.getElementsByTagName("released-by")[0]) if movie.getElementsByTagName("released-by") else None

            user_rating = get_text(movie.getElementsByTagName("rating")[0]) if movie.getElementsByTagName("rating") else None
            rating_support = get_text(movie.getElementsByTagName("support")[0]) if movie.getElementsByTagName("support") else None

            actors = []
            for actor in movie.getElementsByTagName("actor"):
                actor_name = get_text(actor.getElementsByTagName("name")[0]) if actor.getElementsByTagName("name") else None
                actor_role = get_text(actor.getElementsByTagName("role")[0]) if actor.getElementsByTagName("role") else None
                if actor_name and actor_role:
                    actors.append({"name": actor_name, "role": actor_role})

            credits = []
            for credit in movie.getElementsByTagName("credit"):
                credit_name = get_text(credit.getElementsByTagName("name")[0]) if credit.getElementsByTagName("name") else None
                credit_role = get_text(credit.getElementsByTagName("role")[0]) if credit.getElementsByTagName("role") else None
                if credit_name and credit_role:
                    credits.append({"name": credit_name, "role": credit_role})

```

```

review_paragraphs = []
for paragraph in movie.getElementsByTagName("paragraph"):
    paragraph_text = get_text(paragraph)
    if paragraph_text:
        review_paragraphs.append(paragraph_text)

movie_data = {
    "title": title,
    "year": int(year) if year and year.isdigit() else None,
    "rating": rating,
    "country": country,
    "running_time": int(running_time) if running_time and running_time.isdigit() else None,
    "format": format_type,
    "genres": list(genres),
    "production_companies": list(production_companies),
    "released_by": released_by,
    "user_rating": float(user_rating) if user_rating and user_rating.replace(".", "").isdigit() else None,
    "rating_support": int(rating_support) if rating_support and rating_support.isdigit() else None,
    "actors": actors,
    "credits": credits,
    "review": " ".join(review_paragraphs)
}

# store in main dictionary
movie_dict[tv_id] = movie_data

return movie_dict

except Exception as e:
    print(f"Error in parse_tv_guide_ALL: {e}")
    return {} # Return empty dictionary on error

def parse_imdb_complete(file_path):
    handler = IMDBHandlerCompleteALL()
    xml.sax.parse(file_path, handler)
    return handler.get_movie_dict()

def normalize_name(name):
    if isinstance(name, list): # Added this condition to handle the list of actors
        return [normalize_name(actor['name']) for actor in name]
    actor = name.strip()

    if "," in actor:
        parts = [part.strip().capitalize() for part in actor.split(",")]
        actor = f"{parts[1]} {parts[0]}" if len(parts) > 1 else actor.capitalize()
    else:
        actor = " ".join(word.capitalize() for word in actor.split())

    return actor

def merge_lists(list1, list2, key):
    merged_dict = {d[key]: d for d in list1 if key in d} # store first list in a dictionary

    for item in list2:
        if key in item:
            merged_dict[item[key]] = item # overwrite with second list's values

    return list(merged_dict.values())

def merge_movies(tv_guide_movies, imdb_movies, matches_file_path):
    merged_movies = {}

    with open(matches_file_path, 'r') as file:
        matches = file.read()

    match_dict = {}
    for line in matches.strip().split('\n'):
        parts = line.split(',', 2)
        if len(parts) >= 2:
            tv_title = parts[0].strip()
            imdb_title = parts[1].strip()

            # get the score
            score = None
            if len(parts) > 2:
                try:
                    score = float(parts[2])
                except ValueError:

```

```

pass

match_dict[tv_title] = (imdb_title, score)

# Create a title-to-ID mapping for IMDB movies
title_to_id = {movie_data['title']: imdb_id for imdb_id, movie_data in imdb_movies.items()}

# going through each TV guide movie
for tv_id, tv_movie in tv_guide_movies.items():
    if not isinstance(tv_movie, dict):
        print(f"Skipping invalid movie entry: {tv_title}")
        continue

    tv_title = tv_guide_movies[tv_id]['title']
    # find matching IMDB movie
    imdb_id = None
    if tv_title in match_dict.keys():
        imdb_title, _ = match_dict[tv_title]
        imdb_id = title_to_id.get(imdb_title)

    if imdb_id and imdb_id in imdb_movies:
        imdb_movie = imdb_movies[imdb_id]

        # A: Merge single-valued attributes from IMDB (overwrite TV Guide values if different)
        tv_movie['title'] = imdb_movie.get('title', tv_title) # Overwrite title if different
        tv_movie['year'] = str(imdb_movie.get('year', tv_movie.get('year', '')))

        for key in ['director', 'location', 'keywords', 'plot']:
            if imdb_movie.get(key):
                tv_movie[key + 's'] = imdb_movie[key]

        # add IMDB ID
        tv_movie['imdbid'] = imdb_id

        # B: Merge genres (union of genres from both sources)
        tv_genres = tv_movie.get('genres', [])
        imdb_genres = imdb_movie.get('genres', [])

        if isinstance(tv_genres, str):
            tv_genres = [tv_genres]

        all_genres = set(tv_genres) | set(imdb_genres)
        tv_movie['genres'] = list(all_genres)

        # C: Merge actors
        tv_actors = tv_movie.get('actors', {})
        imdb_actors = imdb_movie.get('actors', [])

        if isinstance(tv_actors, dict): # Convert single actor to list
            tv_actors = [tv_actors]

        for actor in imdb_actors:
            if isinstance(actor, dict) and 'name' in actor:
                actor['name'] = normalize_name(actor['name'])

        # dictionary to store normalized actors (to avoid duplicates)
        normalized_actors = {normalize_name(actor['name']): actor for actor in tv_actors if isinstance(actor, dict) and 'name' in actor}

        merged_list = merge_lists(tv_actors, normalized_actors, "role")
        print(merged_list)

        # update actor list in TV movie
        tv_movie['actors'] = merged_list

    # store the merged movie
    merged_movies[tv_title] = tv_movie

return merged_movies

# just to inspect if it works, but unnecessary => inspect JSON file is better
def inspect_sample(merged_movies, sample_size=3):
    print(f"\nSample of {sample_size} merged movies for inspection:")

    movies_list = list(merged_movies.values())[:sample_size]

    for i, movie in enumerate(movies_list):
        print(f"\n--- Movie {i+1} ---")

```

```

print(f"Title: {movie.get('name')}")
print(f"Year: {movie.get('year')}")
print(f"IMDB ID: {movie.get('imdbid', 'Not available')}")

if 'genres' in movie and 'genre' in movie['genres']:
    genres = movie['genres']['genre']
    if isinstance(genres, list):
        print(f"Genres: {' '.join(genres)}")
    else:
        print(f"Genres: {genres}")

if 'directors' in movie:
    print(f"Directors: {movie['directors']}")

if 'locations' in movie:
    print(f"Locations: {movie['locations']}")

if 'actor-list' in movie and 'actor' in movie['actor-list']:
    actors = movie['actor-list']['actor']
    if isinstance(actors, list):
        print(f"Sample actors ({min(3, len(actors))} of {len(actors)}):")
        for actor in actors[:3]:
            if isinstance(actor, dict):
                print(f"  - {actor.get('name', 'Unknown')}: {actor.get('role', 'Unknown role')}")
            else:
                if isinstance(actors, dict):
                    print(f"Actor: {actors.get('name', 'Unknown')}: {actors.get('role', 'Unknown role')}")

# File paths
tv_guide_file = "tvguide-ds.xml"
imdb_file = "imdb-ds.xml"
matches_file = "/content/matches_mapping.txt"

print("Parsing TV Guide data...")
try:
    tv_guide_movies = parse_tv_guide_ALL(tv_guide_file)
    print(f"Successfully parsed {len(tv_guide_movies)} TV Guide movies")
except Exception as e:
    print(f"Error parsing TV Guide data: {e}")
    # for testing, create a sample TV Guide movie
    tv_guide_movies = [ast.literal_eval("""{'name': '88 Minutes',
'rank': '86',
'year': '2008',
'rating': 'R',
'country': 'U.S.',
'running-time': '108',
'format': 'Color',
'genres': {'genre': 'Thriller'},
'production-companies': {'production-company': ['Equity Pictures Medienfonds',
'Millennium Films',
'Nu Image Entertainment']},
'released-by': 'Sony',
'user-rating': {'rating': '3.5', 'support': '10'},
'actor-list': {'actor': [{'name': 'Carrie Genzel',
'role': 'Stephanie Parkman'},
{'name': 'Christopher Redman', 'role': 'Jeremy Guber'},
{'name': 'Al Pacino', 'role': 'Jack Gramm'}}}]""")]

 Parsing TV Guide data...
Successfully parsed 98 TV Guide movies

# Load IMDB data
print("Parsing IMDB data...")
try:
    imdb_movies = parse_imdb_complete(imdb_file)
    print(f"Successfully parsed {len(imdb_movies)} IMDB movies")
except Exception as e:
    print(f"Error parsing IMDB data: {e}")
    # for testing, create a sample IMDB movie
    imdb_movies = {
        'idm123456': {
            'title': '88 Minutes',
            'year': 2008,
            'imdbid': 'idm123456',
            'genres': ['Thriller', 'Crime'],
            'director': 'Jon Avnet',
            'location': 'Seattle, Washington, USA',
            'keywords': ['forensic psychiatrist', 'death row', 'countdown'],

```

```

'plot': 'On the day a serial killer is set to be executed, he claims he is innocent and a forensic psychiatrist has 88 minut
'actors': [
    {'name': 'Al Pacino', 'role': 'Jack Gramm'},
    {'name': 'Alicia Witt', 'role': 'Kim Cummings'},
    {'name': 'Neal McDonough', 'role': 'Jon Forster'}
]
}
}

```

➔ Parsing IMDB data...
Successfully parsed 243856 IMDB movies

```

# merge movies
print("Merging movies based on matches...")
merged_movies = merge_movies(tv_guide_movies, imdb_movies, matches_file)
print(f"Successfully merged {len(merged_movies)} movies")

```

```
# inspect_sample(merged_movies)
```

```

# Save the merged data to a JSON file
with open("merged_movies.json", "w") as f:
    json.dump(merged_movies, f, indent=2)
print("\nMerged data saved to 'merged_movies.json'")

```

➔ Merging movies based on matches...

```

Ever After
[{'name': 'Matyelok Gibbs', 'role': 'Louise'}, {'name': 'Joerg Stadler', 'role': 'Wilhelm Grimm'}, {'name': 'Elvira Stevenson', '
First Daughter
[{'name': 'Forest Whitaker', 'role': 'Narrator'}, {'name': 'Joan Rivers', 'role': 'Herself'}, {'name': 'Michael Connor', 'role':
Hope Floats
[{'name': 'Dee Hennigan', 'role': 'Dot'}, {'name': 'Michael Pare', 'role': 'Bill Pruitt'}, {'name': 'Kathy Najimy', 'role': 'Toni
88 Minutes
[{'name': 'Carrie Genzel', 'role': 'Stephanie Parkman'}, {'name': 'Christopher Redman', 'role': 'Jeremy Guber'}, {'name': 'Tim He
Chicago
[{'name': 'Christine Baranski', 'role': 'Mary Sunshine'}, {'name': 'Sebastian Lacause', 'role': 'June's Husband, Wilbur'}, {'name
All The King's Men
[{'name': 'Patricia Clarkson', 'role': 'Sadie Burke'}, {'name': 'Valerie Stodghill', 'role': 'Miss DuMonde'}, {'name': 'Travis M.
Across The Universe
[{'name': 'Michael Arenella', 'role': 'Member of Circus Band'}, {'name': 'Arabella Holzbog', 'role': 'Prankster'}, {'name': 'Joe
XXX
[{'name': 'Tom Everett', 'role': 'Senator Dick Hotchkiss'}, {'name': 'Thomas Ian Griffith', 'role': 'Agent Jim McGrath'}, {'name'
Quantum Of Solace
[{'name': 'Mathieu Amalric', 'role': 'Dominic Greene'}, {'name': 'Giancarlo Giannini', 'role': 'Rene Mathis'}, {'name': 'Olga Kur
Max Payne
[{'name': 'Olga Kurylenko', 'role': 'Natasha'}, {'name': 'Beau Bridges', 'role': 'B.B. Hensley'}, {'name': 'Chris Bridges', 'role
Blazing Saddles
[{'name': 'Slim Pickens', 'role': 'Taggart'}, {'name': 'John Hillerman', 'role': 'Howard Johnson'}, {'name': 'George Furth', 'rol
Yentl
[{'name': 'Ruth Goring', 'role': 'Esther Rachel'}, {'name': 'Frank Baker', 'role': 'Village Student'}, {'name': 'Kerry Shale', 'r
Changeling
[{'name': 'Peter Gerety', 'role': 'Dr. Earl W. Tarr'}, {'name': 'Angelina Jolie', 'role': 'Christine Collins'}, {'name': 'Frank W
Miss Congeniality
[{'name': 'Lucien Douglas', 'role': 'Warehouse Hair Stylist'}, {'name': 'Summyr Miller', 'role': 'Miss Utah'}, {'name': 'Pei-San
Casino Royale
[{'name': 'Makhoudia Diaw', 'role': 'Obanno's Liaison'}, {'name': 'Michael Offei', 'role': 'Obanno's Lieutenant'}, {'name': 'Dus
Sky High
[{'name': 'Patrick Warburton', 'role': 'Voice of Royal Pain'}, {'name': 'Mary Elizabeth Winstead', 'role': 'Gwen Grayson'}, {'nam
Madagascar: Escape 2 Africa
[{'name': 'Jada Pinkett Smith', 'role': 'Gloria the Hippo'}, {'name': 'Chris Rock', 'role': 'Marty the Zebra'}, {'name': 'will.i.
Stars In My Crown
[{'name': 'Connie Gilchrist', 'role': 'Sarah Isbell'}, {'name': 'Marshall Thompson', 'role': 'Narrator'}, {'name': 'Dean Stockwel
Body Of Lies
[{'name': 'Mehdi Nebbou', 'role': 'Nizar'}, {'name': 'Oscar Isaac', 'role': 'Bassam'}, {'name': 'Ali Suliman', 'role': 'Omar Sadi
Forever Love
[{'name': 'Ryan Browning', 'role': 'Paramedic'}, {'name': 'Evan Taylor', 'role': 'ER Doctor'}, {'name': 'Antonia Jones', 'role':
Black Snake Moan
[{'name': 'John Cothran', 'role': 'Reverend R.L.'}, {'name': 'Neimus K. Williams', 'role': 'Lincoln'}, {'name': 'Claude Phillips'
Tomorrow Never Dies
[{'name': 'Mark Spalding', 'role': 'Stealth Boat Captain'}, {'name': 'Brendan Coyle', 'role': 'Leading Seaman--HMS Bedford'}, {'n
Goldeneye
[{'name': 'Gottfried John', 'role': 'General Ourumov'}, {'name': 'Izabella Scorupco', 'role': 'Natalya Simonova'}, {'name': 'Tche
Slumdog Millionaire
[{'name': 'Freida Pinto', 'role': 'Latika (older)'}, {'name': 'Tanvi Ganesh Lonkar', 'role': 'Latika (middle)'}, {'name': 'Tanay
Raising Arizona
[{'name': 'Dennis Sullivan', 'role': 'Policemen in Arizona House'}, {'name': 'Cody Ranger', 'role': 'Arizona Quint'}, {'name': 'N
Fireproof
[]
Mamma Mia!
[{'name': 'Christine Baranski', 'role': 'Tanya'}, {'name': 'Tommy Franzen', 'role': 'Stag'}, {'name': 'Chris Jarvis', 'role': 'Ed
Indiana Jones And The Kingdom Of The Crystal Skull
[{'name': 'Harrison Ford', 'role': 'Indiana Jones'}, {'name': 'Igor Jijikine', 'role': 'Colonel Dovchenko'}, {'name': 'Cate Blanc

```

